

Bearer Token Authentication

Protecting REST API Endpoints

Why REST APIs Prefer Tokens Over Sessions

Sessions rely on browser cookies and server-side session state. While API clients can technically handle cookies, REST APIs are designed to be stateless.

- Many API clients (cURL, mobile apps, other servers) don't use browser cookie jars.
- APIs need a clean, standard way to authenticate every request independently without relying on cookies.

Bearer Token as the Solution

Bearer token = a string the client sends with each request: "whoever holds this token gets access."

How It's Sent

GET /students/5

Authorization: Bearer john-demo-token

- **Bearer** is the **scheme** — how the token is sent.
- The token itself can be any unique string.

- In other words, the client says: "Here is my proof of identity, and I am sending it in the **Authorization** header."
- The server then checks whether that token is one it issued before or one it trusts.
- This is why you do not send a password again on every request; the token acts like a temporary pass.

| How a Token Is Created and Used

In a real login flow, the server does this:

1. The client sends a username and password.
2. The server verifies the password.
3. If valid, the server creates a token such as `john-demo-token`.

4. The server stores a mapping like `john-demo-token -> john`.
5. The server sends that token back to the client.
6. The client keeps it in memory, localStorage, or a cookie and attaches it to later requests.

Every later API request then looks like this:

```
GET /students/5  
Authorization: Bearer john-demo-token
```

The server checks the token again before allowing access.

- If it is missing, wrong, or expired, the server rejects the request with **401 Unauthorized**.

Extracting and Checking the Token

```
function getBearerToken() {
    $headers = getallheaders();
    if (isset($headers['Authorization']) &&
        preg_match('/Bearer\s+(.*)$/i', $headers['Authorization'], $m)) {
        return trim($m[1]);
    }
    return null;
}

function isValidToken($token) {
    $validTokens = ['john-demo-token' => 'john', 'admin-demo-token' => 'admin'];
    return $validTokens[$token] ?? false;
}
```

| Requiring Auth on a Protected Endpoint

`sendJsonError()` is a small helper.

- It wraps `http_response_code()`, `echo` `json_encode(...)`, and `exit` in one call so every endpoint reports errors the same way.

```
function requireAuth() {  
    $token = getBearerToken();  
    if (!$token) {  
        sendJsonError(401, 'Bearer token required');  
    }  
    $user = isValidToken($token);  
    if (!$user) {  
        sendJsonError(401, 'Invalid or expired token');  
    }  
    return $user;  
}
```

Call `requireAuth()` at the top of any endpoint that needs protection — exactly the pattern Project 1 needs for its two protected endpoints.

Testing Bearer Token

We use two tools: cURL & Javascript

Testing With cURL

```
# No token – rejected (401)
curl http://localhost:8080/protected_api.php
{"error":"Bearer token required"}

# Valid token – allowed (200)
curl -H "Authorization: Bearer john-demo-token" http://localhost:8080/protected_api.php
{"message":"Welcome to the protected API!","authenticated_user":"john", ...}
```

Always test all three cases: **missing** token, **invalid** token, and a **valid** token — Project 1's test files should cover all three too.

Testing With JavaScript

```
fetch('http://localhost:8080/protected_api.php', {  
  headers: { 'Authorization': `Bearer ${token}` }  
})  
  .then(res => res.json())  
  .then(data => console.log(data));
```

Same header, same server-side check — the client just adds one header to a normal `fetch()` call.

| Testing With an Automated Script

Typing `curl` commands by hand works, but a script that runs all three cases every time and tells you pass or fail catches mistakes faster than reading raw JSON.

```
bash ./test_curl.sh
```

`test_curl.sh` runs the whole flow back-to-back:

1. **Login and capture the token** — it parses the token out of the login response instead of hardcoding one, so it always tests with a token the server just issued.
2. **Valid token** — expects `200` and `"authenticated_user":"john"`.
3. **Invalid token** — expects `401` and `"Invalid or expired token"`.
4. **No token at all** — expects `401` and `"Bearer token required"`.

```
run_test() {  
    response=$(curl -s -w "\n%{http_code}" "$@" )  
    status="${response##*$'\n'}"  
    body="${response%$'\n'*}"  
    # compare $status and $body against what this test expects  
}
```

- Same idea as `password_verify()` from last lecture: don't trust output by eye — compare it against a known-correct value.
- A script that checks its own results catches a broken server or a typo in `bearer_auth.php` immediately, instead of a student quietly missing a wrong response.
- Each check prints `PASS` or `FAIL`, and the script exits with a non-zero code if anything failed — the same signal a CI pipeline would look for.

Key Takeaways

1. Bearer tokens authenticate stateless API requests without cookies.
2. The client sends `Authorization: Bearer <token>` on every request.
3. The server extracts the token, checks it, and returns `401` if it's missing or invalid — this is what Project 1's two protected endpoints must do.
4. Test all three cases: missing, invalid, and valid tokens.